

Network Layer

It's one of the most important layers. (In every single internet device, including hosts and routers)

It has 2 parts

1. Data Plane
 - It's about local, per router functions.
 - Decides how the traffic from the input port is forwarded to the output port
2. Control Plane
 - It's a network wide logic.
 - It's the entire view of getting packets from 1 end of the world to the other end.
 - 2 Approaches
 - Traditional Routing Algorithms (in routers)
 - * Each router has a forwarding table (header -> router)(Computed by having a routing algorithm run on every router, to track the time for every other router)
 - Software Defined Networking (SDN) in remote servers.
 - * There's a remote controller that computes, and distributes the forwarding tables in the routers

Has a sender and receiver. Sender sends segments to the link layer. Receiver delivers the segments to the transport layer.

Routers go through all the header fields, and move datagrams from its input ports to the output ports.

Network Layer Functions

- Forwarding: Move packets from a routers own input to output port. Happens in nano seconds, and is implemented via hardware.
- Routing: Determines the route taken by the packet.

Network Layer Service Model

"BEST EFFORT" service model.

No guarantee on:

- Successful datagram delivery to destination
- Timing or order of delivery.
- Bandwidth availability to end-end flow

It's very simple to implement, maintain, and manage.

Most applications run good enough most of the time, in best effort, as there's sufficient provisioning of bandwidth.

Allows distributed services to be provided from multiple locations cuz of best effort.

What's Inside a Router

There's basically a network within a network, and like, there's a processor that controls it all (also acts as the SDN, and sets forwarding tables for all the input ports of the router)

Input Port

Has the Line termination (physical layer), the link layer protocol, and then the networking layer (decentralised switching), that deals with the lookup, forwarding, and queuing.

In the decentralised switching,

1. We'll look at the header field values, and then, lookup the output port using the forwarding table
2. Destination based forwarding: based on the destination IP address
3. Generalised Forwarding: based on a set of header field values

Destination based forwarding

1. Range based approach
2. Longest Prefix Match
 - 10101010 01010101 10101*** ***** = 0
 - 10101010 01010101 10101010 ***** = 1
 - 10101010 01010101 10100*** ***** = 2
 - else = 3

Use the longest address prefix that matches the destination address.

Often performed using ternary content addressable memories (TCAMs). The content gets delivered in 1 clock cycle, regardless of the size of the table

Switching Fabric

Moves the packets from the input port to the output port.

Switching Rate = Rate at which the packets can be transferred from the input port to the output port.

Switching Via Memory

Packet copied to memory, and then, to the output port. So like, the ports were basically like IO devices. Depends on memory bandwidth.

Switching Via Bus

Packet is written directly to the Output port. Depends on bus bandwidth.

Switching Via Interconnection Networks

Made via connecting multistage switching networks in parallel, with smaller switches. switches.

We can exploit parallelism further, by stacking multiple planes together.

Input Port Queuing

If the switching fabric is slower than the input ports combined, queuing may occur at input queues. All the packets that are waiting, wait at the input port.

This is called HOL (head of line) blocking

Output Port Queuing



Read this part again

Why is it bad to have infinite buffering

Bits arrive at , but only leave at . Thus, it requires buffering. So, we'll need a drop policy as well, if the buffer is full. There will also be a scheduling discipline, which could give priority to specific packets.

RFC 3439 says that the amount of buffering should be (link capacity).

A more recent recommendation says, , with N flows crossing the link.

Large buffers = larger delays, and larger RTTs

Buffer Management

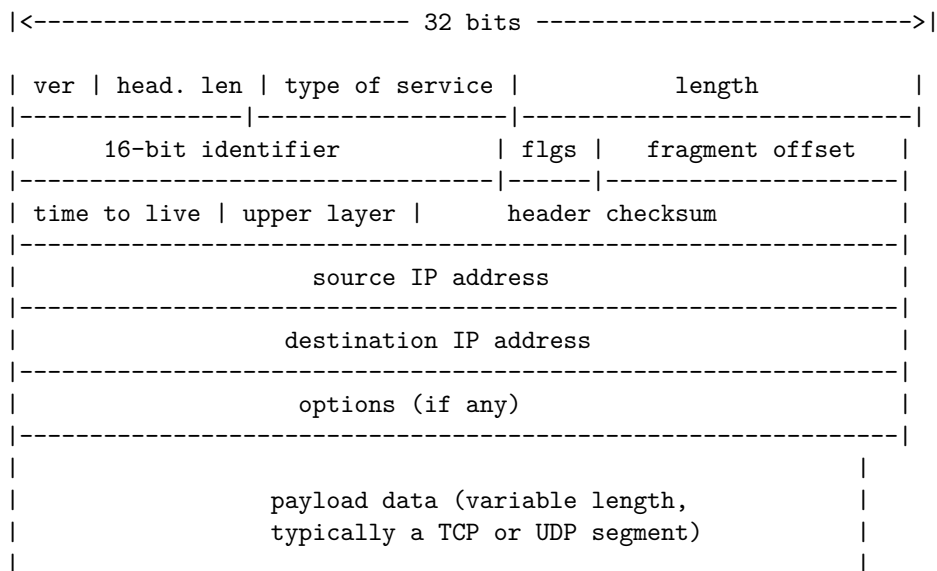
1. Drop: Which packet to add or drop when the buffers are full. Could just drop the last packet (tail dropping), or we could remove the least important packet (priority based dropping)
2. Marking: We can mark some of the packets to show congestion (ECN, RED)

Packet Scheduling

1. FCFS (FIFO)
2. Priority
3. Round Robin
4. Weighted Fair Queuing

The Internet Protocol

IP Datagram format



- Ver = IP version number
- header length (bytes)
- length: total datagram length (bytes)
- max length = 64k bytes. But usually, 1500 bytes or less
- type of service: diffserv(0:5), ECN (6:7) (ECN was for congestion control)
- TTL: the count that gets decremented by 1 each time it passes by a router. If it reaches 0, it must be dropped. Basically ensures that the packet doesn't loop forever, if there are forwarding loops.

- Upper Layer: Upper layer protocol. The protocol in which the IP layer will be passed. 6 = TCP, 17 = UDP
- IP ID, flags, and fragment offset fields are used when a large IP datagram is fragmented into multiple mini datagrams.
- Header Checksum = Checksum

Usually 20 bytes of IP, and 20 bytes for TCP.
So, 40 Bytes + app layer overhead.

IP Addressing

An IP address is unique for every link layer interface on a host or router.

Usually all routers have multiple interfaces, and hosts do to. (eg: Ethernet, 80211 (wifi), etc)

IP address: <8 bits>.<8 bits>.<8 bits>.<8 bits> = 4 bytes

Subnet

Device interfaces that can physically reach each other without passing through an intervening router

And IP address has 2 parts. Subnet part (high order bits), and the host part (low order bits). So, if 2 devices are in the same subnet, they'll have the same subnet part in their IP addr.

CIDR

Classless InterDomain Routing

223.1.1.0/24 = 223.1.1.***** = 24 bits are filled.

a.b.c.d/x, where x is the number of bits in the subnet portion. Sometimes the d is emitted, when it's 0.

How to get an IP Address? DHCP

Used to be hardcoded in /etc/rc.config
now, it's via DHCP.

The network will have a DHCP server. The host will request for an IP address via the DHCP protocol, from the DHCP server. The IP addresses can later be reused too, by another host, or renewed.

DHCP runs over UDP

Client uses port 68, and the server uses port 67

host: DHCP discover msg [optional]

src: 0.0.0.0, 68

dest: 255.255.255.255, 67

transaction id: 654

yiaddr: 0.0.0.0

dhcp: DHCP offer msg [optional]

src: 223.1.2.5, 67

dest: 255.255.255.255, 68

yiaddr: 223.1.2.4

transaction id: 654

lifetime: 3600 seconds

host: DHCP request

src: 0.0.0.0, 68

dest: 255.255.255.255, 67

yiaddr: 223.1.2.4

transaction id: 655

lifetime: 3600 seconds

dhcp: DHCP ACK

src: 223.1.2.5, 67

dest: 255.255.255.255, 68

yiaddr: 223.1.2.4

transaction id: 655

lifetime: 3600 seconds

DHCP also usually returns more stuff

1. Address of the first hop router for the client
2. Name and address of the DNS server
3. The network mask (the number of bits and the ip address belonging to the subnet)

ISP's block: 200.23.16.0/20

ISP can allocate out its address space in 8 blocks

200.23.16.0/23

200.23.18.0/23

...

200.23.30.0/23

Now, ISP-A : 200.23.16.0/20

and, ISP-B : 199.31.0.0/16

What if org1 of ISP-A switches to ISP-B, but with the same address, 200.23.18.0/23

Now, ISP-A: 200.23.16.0/20

and, ISP-B: 199.31.0.0/16 or 200.23.18.0/23

200.23.18.0/23 qualifies for both ISPs, but ISP-B has a longer 23 bit prefix. More specific prefix than the 20 bit prefix in ISP-A. Hence, it'll qualify for ISP-B, and not ISP-A.



How does an ISP get an address space?

The IPv4 space is owned by ICANN (IPv6 is governed by IANA, which is a part of ICANN too). ICANN allocates IP addresses through 5 regional registries (RR's), who then allocate to ISPs.

ICANN also manages root DNS, domain names, and allocating protocol numbers.

In 2011, ICANN allocated the last chunk of IPv4 addresses to RRs

NAT (Network Address Translation)

All devices in a local network share just 1 IPv4.

All devices in a local network have 32 bit addresses (private IP), 10/8, 172.16/12, 192.168/16



Now, if A sends a req to B, B will get the IP of the router that A's connected to. Then, like, when it sends the response back, how will the router know which device is A from all the devices connected to it?

A sends

src: 192.168.1.5:1234

dst: 8.8.8.8:80

Router modifies it

src: 45.10.20.30:5678

dst: 8.8.8.8:80

Router stores a mapping

45.10.20.30:5678 192.168.1.5:1234

B sends

dst: 45.10.20.30:5678

Router checks

5678 192.168.1.5:1234

so, it modifies it to

src: 8.8.8.8:80

dst: 192.168.1.5:1234

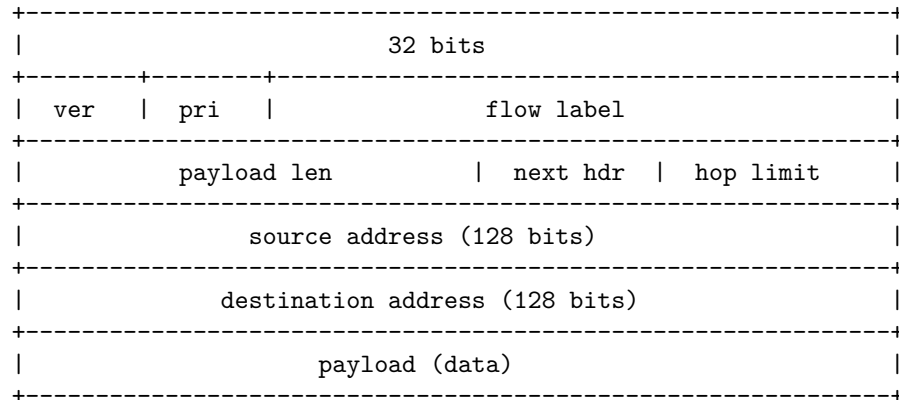
IPv6

32 bit ipv4 128 bit ipv6

ipv6 also has additional features like, faster processing, and better forwarding.

A 40 byte fixed length header

ipv6 abstracted datagrams into flows



- flow label: identify datagrams in the same flow (not properly defined...up to the ISP on how to do stuff with it)
- 128 bit addresses
- pri = priority (8 bit)

There's NO checksum, TTL, fragmentation/reassembly, or options in ipv6. It makes the header of a fixed length, and allows for faster processing

Checksum is removed as other layers already have error checking, and the checksum ends up happening at every router hop, which required it to be recomputed at every hop, making it slow. Router's also have to modify the TTL field, which also takes time. In ipv6, udp checksum is also mandatory, whereas in ipv4, it's not, and can just be 0x0000.

Fragmentation and reassembly needs to be done at the end points, and the options can be sent as an IPv6 datagram payload upto an upper layer protocol at the router (responsibility of the sender)

Tunnelling

There are different kinds of routers. IPv6 only, IPv4 only, and ones who support both. Now basically, IPv6 and IPv4 only routers can't communicate directly with each other.

Suppose, A wants to send a packet to F (both are IPv6).

- A, B, E, F have IPv6 addresses
 - B, C, D, E have IPv4 addresses
1. At A, ipv6 forwarding table tells it to go to B
 2. At B, ipv6 forwarding table tells it to tunnel to E, the ipv4 forwarding table tells it to go to C

3. At C, ipv4 forwarding table tells it to go to D
4. At D, ipv4 forwarding table tells it to go to E
5. At E, ipv6 forwarding table tells it to go to F

Tunnelling = [IP v4 packet [IP v6 packet]]

IPv6 packet gets encapsulated within the IPv4 packet.